



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251189
Nama Lengkap	Alicia Luna Santoso
Minggu ke / Materi	12/ Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Link Github : https://github.com/alc357/71251189_alicia.git

MATERI

Dictionary mirip dengan list, tetapi sifatnya lebih umum. Dictionary memiliki anggota yang terdiri dari pasangan kunci:nilai. Kunci harus bersifat unik, tidak boleh ada dua kunci sama dalam satu dictionary. Dictionary dapat diartikan pula sebagai pemetaan antara sekumpulan indeks (disebut kunci) dan sekumpulan nilai. Setiap kunci memetakan suatu nilai. Asosiasi nilai dan kunci ini disebut pasangan nilai kunci (key-value pair) atau ada yang menyebutnya sebagai item. Contoh, kita akan mengembangkan kamus yang memetakan dari kata-kata dalam Bahasa Inggris ke Bahasa Spanyol, jadi bisa dikatakan yang menjadi kunci dan nilai adalah data string. Asumsinya setiap kata dalam Bahasa Inggris mempunyai satu arti dalam Bahasa Spanyol.

Fungsi dict digunakan untuk membuat dictionary baru yang kosong. Karena dict adalah built-in-function dari python, maka penggunaannya perlu dihindari sebagai nama variabel.

```
>>> ens2sp = dict()
>>> print(ens2sp)
{} _
```

Tanda kurung kurawal { } digunakan untuk merepresentasikan dictionary kosong. Untuk menambah item dalam dictionary, dapat menggunakan kurung kotak []

```
>>> eng2sp['one'] = 'uno'
```

Jika diperhatikan, dapat terlihat bahwa baris tersebut membuat item yang memetakan dari kunci 'satu' ke nilai 'uno'. Dan jika kita mencetak dictionary tersebut, maka akan terlihat padangan nilai kunci antara kunci dan nilai.

```
>>> print(eng2sp)
{'one': 'uno'}
```

Format output yang digunakan adalah format input. Misalkan kita buat dictionary baru dengan item dan melakukan pencetakan hasil yang didapatkan berupa keseluruhan data dari dictionary itu

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Pengurutan pasangan kunci-nilai tidak sama. Secara umum, urutan item pada dictionary tidak dapat diprediksi. Namun ini tidak menjadi masalah utama karena elemen dalam dictionary tidak pernah diberikan indeks dengan indeks integer. Gantinya, dapat menggunakan kunci untuk mencari nilai yang sesuai (corresponding values)

```
>>> print(eng2sp['two'])
dos
```

Kunci two selalu dipasangkan dengan nilai “dos” jadi urutan pada item tidak terlalu berpengaruh.

Jika menggunakan kunci yang tidak ada dalam dictionary, akan didapatkan pengecualian :

```
>>> print(eng2sp['four'])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    import platform
    ^^^^^^^^^^^
KeyError: 'four'
```

Fungsi len pada dictionary digunakan untuk mengembalikan jumlah pasangan nilai kunci :

```
>>> len(eng2sp)
3
```

Operator in dalam dictionary mengemnaliikan nilai benar (true) atau salah (false) sesuai dengan kunci yang ada dalam dictionary

```
>>> 'one' in eng2sp
True
>>> 'uno' in eng2sp
False
```

Untuk mengetahui nilai yang ada pada dictionary dapat menggunakan method values yang akan mengembalikan nilai sesuai dengan tipe datanya dan dikonversi dalam list dan digunakan dalam operator in.

```
>>> vals = list(eng2sp.values())
>>> 'uno' in vals
True
```

Operator in menggunakan algoritma yang berbeda untuk list dan dictionary. Dalam list, menggunakan algoritma pencarian linear dimana saat list bertambah, waktu pencarian yang dibutuhkan lebih banyak sesuai dengan panjang list. Sedangkan dictionary, python menggunakan algoritma bernama hash table dengan property yang luar biasa, operator in membutuhkan waktu yang sama untuk memproses tanpa mempedulikan banyak item dalam dictionary.

DICTIONARY SEBAGAI SET PENGHITUNG (COUNTERS)

Contohnya kita diberikan sebuah string dan harus menghitung banyak huruf yang ada. Acaranya adalah :

1. Membuat 26 variabel untuk tiap huruf pada alfabet, lalu memasukkannya dalam string untuk setiap karakter, menambah perhitungan yang sesuai dan menggunakan kondisional berantai.
2. Membuat list dengan 26 elemen kemudian melakukan konversi setiap karakter menjadi angka dengan menggunakan fungsi bawaan, kemudian menggunakan angka sebagai indeks dalam list dan menambah perhitungan yang sesuai.
3. Membuat dictionary dengan karakter sebagai kunci dan perhitungan sebagai nilai yang sesuai, karakter dapat ditambahkan item ke dalam dictionary dan kemudian ditambahkan nilai dari item yang ada.

Berdasarkan 3 pilihan ini, kita akan melakukan perhitungan yang sama, namun dari setiap opsi menerapkan proses perhitungan dengan cara yang berbeda-beda.

Implementasi dilakukan dengan menggunakan perhitungan (computation), beberapa perhitungan biasanya lebih baik dari model perhitungan lainnya. Berikut model penerapannya :

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     if c not in d:
5         d[c] = 1
6     else:
7         d[c] = d[c] + 1
8 print(d)
```

Model ini disebut histogram, yang merupakan istilah statistika dari set perhitungan atau frekuensi. Perulangan for akan melewati string. Setiap kali terjadi looping, jika karakter c tidak ada dalam dictionary maka akan dibuat item baru dengan kunci c dan nilai awal 1, diasumsikan telah muncul sekali. Jika c sudah ada di dictionary maka otomatis akan melakukan penambahan d(c)

Berikut outputnya

```
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Histogram output menunjukkan bahwa huruf “a” dan “b” muncul sekali lalu “o” muncul dua kali, dan seterusnya.

Dictionary memiliki metode yang disebut get untuk mengambil kunci dan nilai default. Jika kunci muncul di dictionary, akan mengembalikan nilai yang sesuai, jika tidak maka akan mengembalikan nilai default. Berikut contohnya :

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Penggunaan get dapat dilakukan untuk menulis loop histogram secara lebih ringkas. Metode get secara otomatis menangani case dimana kunci tidak ada dalam dictionary, dengan menghilangkan if, dapat diringkas dari empat baris menjadi satu baris saja.

```
1 word = 'brontosaurus'
2 d = dict()
3 for c in word:
4     d[c] = d.get(c,0) + 1
5 print(d)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL  Py

```
PS C:\prak_alpro\random> & "C:/Users/alc_357/AppData/Local/Programs/Python/Python39-64/Python.exe" c:/prak_alpro/random/coba.py
{'b': 1, 'r': 2, 'o': 2, 'n': 1, 't': 1, 's': 2, 'a': 1, 'u': 2}
```

Penggunaan metode get untuk menyederhakan loop penghitungan akhirnya menjadi idiom yang umum digunakan dalam python. Dibandingkan loop yang menggunakan pernyataan if dan operator in dengan loop menggunakan metode get melakukan hal yang persis sama, tetapi yang lebih ringkas.

DICTIONARY DAN FILE

Salah satu kegunaan umum dictionary yaitu menghitung kemunculan kata-kata dalam file dengan beberapa teks tertulis. Sebagai contoh, berikut digunakan versi teks singkat dan disederhanakan tanpa tanda baca. Selanjutnya akan digunakan teks adegan dengan tanda baca yang disertakan.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Kita mencoba menulis program Python untuk membaca baris file, memecah setiap baris menjadi daftar kata, kemudian melakukan perulangan melalui setiap kata dalam baris dan menghitung setiap kata menggunakan dictionary.

Ketika menggunakan dua for untuk perulangan, perluasan bagian luar untuk membaca baris file dan perulangan dalam melakukan iterasi melalui setiap kata pada baris tertentu. Perulangan bagian dalam melakukan eksekusi di semua iterasi setiap kali perulangan bagian luar membuat iterasi tunggal. Di bagian ini, perulangan bagian dalam iterasi "lebih cepat", sedangkan perulangan bagian luar iterasi lebih lambat. Kombinasi dua perulangan bersarang memastikan ada proses perhitungan setiap kata pada setiap baris file input

```

1  fname = input('Enter the file name: ')
2  try:
3      fhand = open(fname)
4  except:
5      print('File cannot be opened:', fname)
6      exit()
7  counts = dict()
8  for line in fhand:
9      words = line.split()
10     for word in words:
11         if word not in counts:
12             counts[word] = 1
13         else:
14             counts[word] += 1
15 print(counts)

```

Di bagian statement else, digunakan model penulisan yang lebih singkat untuk menambahkan variable. Ketika menjalankan program, akan didapat data dump jumlah semua dalam urutan hash yang tidak disortir.

```

python count1.py
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1,
'is': 3, 'through': 1, 'pale': 1, 'yonder': 1,
'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1,
'window': 1, 'sick': 1, 'east': 1, 'breaks': 1,
'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1,
'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}

```

LOOPING DAN DICTIONARY

Dalam statement for, dictionary akan bekerja dengan menelusuri kunci yang ada di dalamnya.

Looping akan melakukan pencetakan setiap kunci sesuai dengan hubungan nilainya

```

counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    print(key, counts[key])

```

```

chuck 1
annie 42
jan 100

```

Output menunjukkan kunci tidak berada dalam pola urutan tertentu. Pola ini diimplementasikan dengan berbagai idiom looping yang sudah dideskripsikan pada bagian sebelumnya. Jika ingin menemukan semua entri dalam dictionary dengan nilai diatas 10, maka kode programnya sebagai berikut :

```

counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    if counts[key] > 10 :
        print(key, counts[key])

```

ADVANCED TEXT PARSING

Di bagian ini kita akan menggunakan file dengan tanda baca lengkap.

```
But, soft! what light through yonder window breaks?  
It is the east, and Juliet is the sun.  
Arise, fair sun, and kill the envious moon,  
Who is already sick and pale with grief,
```

Kita dapat menggunakan split dimana fungsi ini akan mencari spasi dan mengubah kata sebagai token yang dipisah oleh spasi. Pemisahan ini akan dibedakan dalam dictionary. Penggunaan huruf kapital dan tidak juga akan dianggap sebagai kata yang berbeda. Model penyelesaian lain dapat juga dengan menggunakan method string lain seperti lower, punctuation dan translate. Jika diperhatikan method string translate merupakan method string yang paling subtle (hampir tidak ketara). Berikut ini merupakan dokumentasi dari method translate. Ganti karakter pada fromstr dengan karakter pada posisi yang sama dengan tostr dan hapus semua karakter yang ada dalam deletetr. Untuk fromstr dan tostr dapat berupa string kosong dan untuk parameter pada deletetr dapat dihilangkan. Kita tidak akan menentukan tostr tetapi kita akan menggunakan parameter deletetr untuk menghapus semua tanda baca. Secara otomatis Python akan memberitahu bagian mana yang dianggap sebagai "tanda baca".

```
>>> import string  
>>> string.punctuation  
'!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~'  
  
import string  
  
fname = input('Enter the file name: ')  
try:  
    fhand = open(fname)  
except:  
    print('File cannot be opened:', fname)  
    exit()  
  
counts = dict()  
for line in fhand:  
    line = line.rstrip()  
    line = line.translate(line.maketrans('', '', string.punctuation))  
    line = line.lower()  
    words = line.split()  
    for word in words:  
        if word not in counts:  
            counts[word] = 1  
        else:  
            counts[word] += 1  
  
print(counts)  
  
Enter the file name: romeo-full.txt  
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2,  
'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1,  
a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40,  
'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1,  
'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}
```

Sumber : Modul 12-Dictionary

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/alc357/71251189_alicia.git

SOAL 1

Source Code :

```
inputdct = input("dictiionary : ")
dct = eval(inputdct)
print("key    Value    item    ")
no = 1
for a in dct :
    print(f"{a}        {dct[a]}        {no}")
    no += 1
```

Langkah – langkah :

1. Pertama, kita buat inputan user untuk memasukkan dictionary
2. Kemudian, kita konversi string inputan menjadi objek dictionary menggunakan fungsi `eval()`, lalu menyimpannya da;am variabel `dct`
3. Selanjutnya, kita mencetak header sebagai keterangan Key Value dan item menggunakan `print()`
4. Kita inisialisasi variabel `no` dengan nilai 1 sebagai counter untuk nomor urut item
5. Kita buat perulangan `for` untuk mengakses setiap key atau `a` yang ada dalam dictionary `dct`

6. Di dalam perulangan, kita cetak nilai key(a) balue dari dictionary yaitu dct[a] dan nomor urut no
7. Setelah mencetak, kita tambahkan no dengan 1 untuk menambah nomor urut pada iterasi berikutnya
8. Perulangan berlanjut ke key berikutnya dalam dictionary sampai semua key selesai diproses

Output :

```
dictiionary : {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
key  Value  item
1    10     1
2    20     2
3    30     3
4    40     4
5    50     5
6    60     6
```

Gambar 2.1 : Output program soal nomor 1

```
inputdct = input("dictiionary : ")
dct = eval(inputdct)

print("key  Value  item  ")

no = 1
for a in dct :
    print(f"{a}      {dct[a]}      {no}")
    no += 1
```

Gambar 2.2 : Kode Program soal nomor 1

SOAL 2

Source Code :

```
lst1 = eval(input("List key : "))
lst2 = eval(input("List Value : "))

dct = {}

for i in range (len(lst1)) :

    dct[lst1[i]] = lst2[i]

print(dct)
```

Langkah – langkah :

1. Pertama, kita buat inputan dari user dengan pesan “List key : “ untuk memasukan list yang nantinya menjadi key dalam dictionary, lalu mengkonversinya menjadi list menggunakan fungsi eval(), kemudian menyimpannya ke dalam variabel lst1
2. Kedua, kita buat inputan dari user dengan pesan “List Value : “ untuk memasukan list yang nantinya menjadi value dalam dictionary, lalu mengkonversinya menjadi list menggunakan fungsi eval(), kemudian menyimpannya ke dalam variabel lst2
3. Selanjutnya, kita buat dictionary kosong dengan nama dct menggunakan kurung kurawal {}
4. Kemudian kita buat perulangan for dengan variabel i yang berjalan dari 0 sampai panjang list1-1 menggunakan fungsi range len(lst1)
5. Berikutnya, dalam perulangan kita mengisi dictionary dct dengan key dari lst1[i] dan value dari lst2[i] menggunakan sintaks dct [lst1[i] = lst2[i]
6. Terakhir, setelah perulangan selesai, kita cetak isi dictionary dct menggunakan fungsi print

Output :

```
List key : ['red', 'green', 'blue']
List Value : ['#FF0000', '#008000', '#0000FF']
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

Gambar 2.3 : Output dari soal nomor 2

```
lst1 = eval(input("List key : "))
lst2 = eval(input("List Value : "))

dct = {}
for i in range (len(lst1)) :
    dct[lst1[i]] = lst2[i]

print(dct)
```

Gambar 2.4 : Kode soal nomor 2

SOAL 3

Source Code :

```
fname = input("Nama File : ")

try :

    fhandle = open(fname)

except FileNotFoundError :

    print("File tidak bisa dibuka:", fname)

    exit()

jumlah = dict()

for line in fhandle :

    if line.startswith("From ") :

        words = line.split()

        email = words[1]

        if email not in jumlah :

            jumlah[email] = 1

        else :

            jumlah[email] += 1

print(jumlah)
```

Langkah – langkah :

1. Pertama, kita minta input dari user berupa nama file
2. Kedua, kita buat struktur try-except untuk menangani kemungkinan file tidak ada. Jika tidak bisa dibuka, maka akan menampilkan pemberitahuan bahwa file tidak bisa dibuka beserta nama file nya, lalu program berhenti dengan exit()
3. Ketiga, dalam blok try, kita buka file menggunakan fungsi open() dengan parameter fname dan menyimpannya ke variabel fhandle

4. Keempat, jika file tidak ditemukan, kita menampilkan pemberitahuan bahwa file tidak ditemukan dan kita hentikan program menggunakan fungsi `exit()`
5. Kelima, kita buat dictionary kosong dengan nama jumlah menggunakan kurung kurawal
6. Berikutnya, kita buat perulangan `for` untuk membaca setiap baris dalam file , di perulangan itu kita membuat kondisi `if` untuk mencari baris yang dimulai dengan "From " menggunakan metode `line.startswith("From ")`
7. Selanjutnya, Jika baris dimulai dengan "From " , maka kita memecah baris tersebut menjadi list kata menggunakan metode `split()` dan menyimpannya dalam variabel `words`
8. Kemudian, kita ambil Alamat email yang ada di indeks 1 (kata kedua) dari list `words` dan menyimpan dalam variabel `email`
9. Setelah itu, kita buat kondisi `if-else` untuk mengecek apakah email sudah ada di dictionary jumlah atau belum
10. Jika belum, maka kita akan menambah email tersebut sebagai key dengan nilai 1 namun jika sudah ada maka kita menambah nilai atau value nya dengan 1
11. Setelah perulangan selesai, kita cetak isi dictionary jumlah menggunakan fungsi `print`

Output :

```
Nama File : mbox-short.txt
{'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlowe@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagnermr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

Gambar 2.5 : Output soal no 3

```

fname = input("Nama File : ")

try :
    fhandle = open(fname)
except FileNotFoundError :
    print("File tidak bisa dibuka:", fname)
    exit()

jumlah = {}
for line in fhandle :
    if line.startswith("From ") :
        words = line.split()
        email = words[1]
        if email not in jumlah :
            jumlah[email] = 1
        else :
            jumlah[email] += 1
print(jumlah)

```

Gambar 2.6 : Kode Program soal nomor 3

SOAL 4

Source Code :

```

fname = input("masukan nama file : ")

try :

    fhandle = open(fname)

except FileNotFoundError :

    print("File tidak bisa dibuka : ", fname)

    exit()

domain_dct = {}

for line in fhandle :

    if line.startswith("From ") :

        kata = line.split()

        email = kata[1]

        domain= email.split('@')[1]

        if domain not in domain_dct :

```

```

        domain_dct[domain] = 1

    else :

        domain_dct[domain] += 1

print(domain_dct)

```

Langkah – langkah :

1. Pertama, kita minta input dari user berupa nama file
2. Kedua, kita buat struktur try-except untuk menangani kemungkinan file tidak ada. Jika tidak bisa dibuka, maka akan menampilkan pemberitahuan bahwa file tidak bisa dibuka beserta nama file nya, lalu program berhenti dengan exit()
3. Ketiga, dalam blok try, kita buka file menggunakan fungsi open() dengan parameter fname dan menyimpannya ke variabel fhandle
4. Keempat, jika file tidak ditemukan, kita menampilkan pemberitahuan bahwa file tidak ditemukan dan kita hentikan program menggunakan fungsi exit()
5. Kelima, kita buat dictionary kosong dengan nama domain_dct menggunakan kurung kurawal
6. Berikutnya, kita buat perulangan for untuk membaca setiap baris dalam file , di perulangan itu kita membuat kondisi if untuk mencari baris yang dimulai dengan "From " menggunakan metode line.startswith("From ")
7. Selanjutnya, Jika baris dimulai dengan "From " , maka kita memecah baris tersebut menjadi list kata menggunakan metode split() dan menyimpannya dalam variabel kata
8. Kemudian, kita ambil Alamat email yang ada di indeks 1 (kata kedua) dari list kata dan menyimpan dalam variabel email
9. Setelah itu, kita ambil nama domain dari Alamat email dengan memisahkan email dengan metode split('@') yang menghasilkan list 2 bagian, lalu kita ambil indeks 1 nya kemudian menyimpan dalam variabel domain

10. Berikutnya, kita membuat kondisi if else untuk mengecek apakah domain sudah ada dalam dictionary domain_dct atau belum
11. Jika belum, maka kita akan menambah domain tersebut sebagai key dengan nilai 1 namun jika sudah ada maka kita menambah nilai atau value nya dengan 1
12. Setelah perulangan selesai, kita cetak isi dictionary jumlah menggunakan fungsi print ()

Output :

```
masukan nama file : mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
```

Gambar 2.7 : Output soal no 3

```
fname = input("masukan nama file : ")

try :
    fhandle = open(fname)
except FileNotFoundError :
    print("File tidak bisa dibuka : ", fname)
    exit()

domain_dct = {}
for line in fhandle :
    if line.startswith("From ") :
        kata = line.split()
        email = kata[1]

        domain= email.split('@')[1]
        if domain not in domain_dct :
            domain_dct[domain] = 1
        else :
            domain_dct[domain] += 1

print(domain_dct)
```

Gambar 2.8 : Kode Program soal nomor 3